PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

umhackathon@um.edu.my

© UMHackathon 2026

Email: umhackathon@um.edu.my

Table of Contents

1. Project Overview 1
2. Background & Business Objective 1
3. Product Purpose 2
4. System Functionalities 2
5. User Stories & Use Cases 3
6. Features Included (Scope Definition) 3
7. Features Not Included (Scope Control) 3
8. Assumptions & Constraints 4
9. Risks & Questions Throughout Development 4

© UMHackathon 2026

Email: umhackathon@um.edu.my

Project Name: Intelligent Supply Chain Agent

Version: 1.0

1. Project Overview

- **Problem Statement:**
  Cross-border e-commerce sellers, particularly Small-and-Medium Enterprises, face highly fragmented procurement demands daily (request arrives through informal channels like chat or email). Existing supply chain tools are disconnected, manual and non-intelligent. Manually extracting data, comparing suppliers across different platforms, calculating shipping costs and landed costs is slow, error-prone and prevents businesses from scaling.
- **Target Domain:** Cross-border e-commerce supply chain automation, AI-native sourcing and procurement, intelligent multi-agent decision systems.
- **Proposed Solution Summary:** Intelligent Supply Chain Agent is a multi-agent collaboration system. It processes unstructured inputs (PDF quotes, plain text requests) and converts them into structured procurement actions. The system is divided into three specialized agents which are (1) Ingestion Agent (Extracts product requirements from text or images), (2) Sourcing Agent (Search supplier databases or APIs, calculates total costs including exchange rates and tariffs and rank the best option), and (3) Execution Agent (Drafting and sending professional inquiries and purchase order to suppliers.)

2. Background & Business Objective

- **Background of the Problem:** For years, procurement has followed a linear, manual path: an employee describes a need in casual language, a buyer decodes the intent, manually compares offers from emails or static spreadsheets, and judges supplier reliability by gut feeling. When that buyer leaves, the hard-earned knowledge about which vendors are

© UMHackathon 2026
Email: umhackathon@um.edu.my

trustworthy walks out the door. No intelligent system captures the decision logic, so the same errors keep occurring.

- **Importance of Solving This Issue:** Reducing the sourcing cycle from hours to seconds allows companies to respond faster to internal demand and avoid losses from fraud or logistics delays. A junior staff member or a founder with no supply-chain experience can immediately work like a seasoned buyer. The business gains a scalable, repeatable way to vet suppliers that improves with every request.
- **Strategic Fit / Impact:** Intelligent Supply Chain Agent aligns with the border shift toward AI-native operational tools in e-commerce. By orchestrating state and decision branches through a dedicated multi-agent pipeline, the system moves beyond simple automation scripts or chatbots. It provides a reliable framework for intelligent workflow orchestration in environments where information is scattered and conditions change frequently, such as cross-border supply chains.

3. Product Purpose

3.1. Main Goal of the System: To provide an automated procurement assistant that ingests fragmented supply chain inputs and produces actional sourcing recommendations with minimal human intervention.

3.2. Intended Users (Target Audience):

- Cross-border e-commerce sellers
- Purchasing departments of trading companies
- Small-Medium-Enterprises owners managing overseas suppliers
- Dropshipping entrepreneurs

4. System Functionalities

4.1. Description: The system operates as a stateful multi-agent workflow engine. Rather than following a fixed linear script, the workflow maintains a shared state object that travels from one

© UMHackathon 2026

agent to the next. This design allows the system to branch or loop based on intermediate outcomes. For instance, if the Sourcing Agent finds no available inventory, workflow can route to the Exception Agent instead of simply returning an error.

4.2. Key Functionalities:

- **Multi-Modal Requirement Extraction (Ingestion Agent):** Accepts text descriptions, and PDF documents. Uses vision-language models to identify product names, quantities, target prices, and delivery deadlines. Output is a structured JSON object used by downstream agents.
- **Supplier Discovery and Cost Calculation (Sourcing Agent):** Queries the connected supplier database or simulated endpoints to find matching products. Applies strict filters on maximum price, lead time, and minimum order quantity. Automatically flags and removes bids that fall below 50% of the market average price to prevent fraud or quality risks. Calculates geographic risk based on the supplier's country relative to the user's location. Returns a ranked shortlist with a designated winner, or a "no_match" status if no supplier meets all criteria.
- **Exception Handling and Recovery (Exception Agent):** Takes the Sourcing Agent's results and produces a professional email draft addressed to the chosen supplier. If no supplier was matched, the agent automatically falls back to a general Request for Information (RFI) email or a summary report, ensuring the user always receives a usable draft ready for review and sending.
- **End-to-end Transparency and Human Approval:** Every agent decision (parsed items, match scores, price-sanity flags, risk alerts, and the final email) is exposed in a web dashboard. The user reviews the recommendation and draft, then confirms or overrides before any real-world communication is sent.

4.3. AI Model & Prompt Design

4.3.1. Model Selection: We use Z.AI GLM (General Language Model) as a unified reasoning core. GLM was selected for three reasons:

- Enterprise-grade entity extraction. It reliably pulls structured fields (item name, quantity, timeline, special conditions) from raw, often incomplete business messages, outperforming hard-coded regexes.
- Multi-step reasoning with tool use. GLM handles the chained, conditional logic required by the re-sourcing loop and can emit instructions for external tool calls (e.g., database lookup) within a single orchestration call.
- Deterministic JSON output compatibility. The model consistently generates well-formed structured output that can be directly written into the State Pool, reducing integration fragility.

4.3.2. **Prompting Strategy:** We deploy a multi-shot, agentic prompting strategy with chain-of-thought. Each agent receives a dedicated system prompt containing (1) Role and business objective, (2) Strict JSON schema for output and (3) Curated few-shot examples that illustrate successful parsing, sourcing justification, and risk evaluation.

4.3.3. **Context & Input Handling**: The system accepts direct text input and uploaded documents (.txt, .docx, .pdf). For documents, text is first extracted using python-docx, PyMuPDF, or a combination of GLM-4V and Tesseract OCR for images. There is no hard character limit。 The user's full input is passed directly into the GLM-4 prompt. If the text is extremely long, it may naturally hit the model's context window, in which case the API call will fail and return an error message to the user.

4.3.4. **Fallback & Failure Behavior:** If an agent's GLM-4 call fails (e.g., network error, model overload), the agent returns a minimal valid JSON object so the pipeline does not crash entirely. For example, if Agent 1 (Ingestion Agent) fails, the system still treats the raw user input as a single product item. If Agent 2 (Sourcing Agent) finds no supplier matches, it reports "no_match" instead of an error. If Agent 3 (Excecution Agent) sees no winners, it automatically

© UMHackathon 2026

generates a generic RFI email. The web interface displays the partial output clearly, so the user always knows what happened.

5. User Stories & Use Cases

- **User Stories:** "As a procurement officer, I want to paste a colleague's chat message like 'need 50 silk hijab under RM200 within 14 days' and instantly see a recommended supplier with price, risk warning, and a draft email – so I don't spend 30 minutes in spreadsheets."
- **Use Cases (Main Interactions):**

  Prompt to Recommendation Flow: User enters free-form text → Agent 1 parses structured items → Agent 2 searches, filters, and risks-checks suppliers → Agent 3 drafts an email → The dashboard shows the winning supplier, risk alerts, and the email draft.

  Document Upload Flow: User uploads a .pdf or text docs → server extracts text using OCR + GLM-4V → the extracted text feeds into the same 3-agent pipeline.

  No Match Handling: If Agent 2 cannot find a suitable supplier, the dashboard shows a "no_match" status and Agent 3 still produces a general inquiry email, so the user can move forward manually.

6. Features Included (Scope Definition)

- Multi-agent pipeline (extract → source → draft) with full JSON visibility.
- Natural-language requisition parsing for English, Malay, and Chinese.
- Fuzzy supplier matching via exact, token-overlap, Malay alias, and category fallback.
- Price-sanity filter automatically excluding bids below 50% of the matched-group average.
- Geographic risk detection tied to Kuala Lumpur (typhoon, monsoon, landlocked flags).

© UMHackathon 2026
Email: umhackathon@um.edu.my

- Professional email draft generation with unconditional fallback.
- Web dashboard displaying live agent output, winner cards, and email previews.
- Supplier CSV upload via drag-and-drop with hot-reload.
- Document intake support for .txt, .docx, .pdf

7. Features Not Included (Scope Control)

- Real-time supplier inventory or live ERP connections (data is a static CSV).
- Cross-border cost calculation, customs, or multi-currency conversion (architecture
- reserves an interface but it is inactive).
- Video or audio input.
- Live scraping of public marketplaces like Shopee.
- Financial transactions, order placement, or contract generation.
- Multi-user simultaneous collaboration in the current version.

8. Assumptions & Constraints

- **LLM Cost Constraint:** A typical user session involves 2–3 language model calls (parsing, email drafting). Estimated token consumption per session is approximately 3,000–4,500 tokens. To keep costs predictable at scale, the price-sanity threshold is calculated mathematically before any model involvement, and fuzzy matching runs entirely on-CPU using string comparison logic.
- **Technical Constraints:**
    - Supplier data is loaded from a CSV file; column names are mapped dynamically, but the file must be manually uploaded or updated.
    - Email drafts are generated as text in the web dashboard; actual sending is simulated and not connected to a real SMTP server.
    - The system does not integrate live currency conversion, shipping tariff APIs, or any external tool-calling endpoints. All cost estimates, risk assessments, and rankings are based on static data and mathematical rules within the application code.

© UMHackathon 2026

Email: umhackathon@um.edu.my

- **Performance Constraints:**
  - Pipeline latency depends on model API response time and supplier matching computation. There is no hard-coded time-out or guaranteed response window.
  - Supplier matching processes all available records without an artificial cap on candidates. Performance scales with the size of the uploaded CSV.
  - Each model generation request does not set an explicit maximum tokens, so the service's default output limit applies, naturally bounding response size.
- **User Input:**
  - The system must receive a human-initiated request; it cannot start procurement on its own.
  - Final purchase approval remains a human decision – the system recommends and drafts communication, but the user always confirms.

9. Risks & Questions Throughout Development

- **Output Similarity:** Different users submitting similar requisitions (e.g. "50 Silk Hijab under RM200") may receive identical supplier recommendations if they are using the same CSV file. The system currently has no personalisation layer or randomised tie-breaking that would introduce variety across sessions. How can we ensure that frequent users do not perceive the output as repetitive or templated?
- **Pipeline Stability:** The three-agent pipeline depends on sequential model calls. If the AI service becomes unreachable after Agent 1 but before Agent 3, the system returns a partially complete JSON stub. The web dashboard displays whatever was produced, but there is no explicit "pipeline interrupted" flag for the user. What visual cues should indicate an incomplete pipeline versus a successful one?
- **Drawing Interpretation**: When a user uploads a PDF that contains only scanned images without embedded text, the extraction libraries (fitz/pdfminer) return an empty string. The pipeline then receives a blank request and will produce either an error or an unusable

© UMHackathon 2026

output. Should the system detect this pre-prompt and immediately inform the user that the document was unreadable?

© UMHackathon 2026

Email: umhackathon@um.edu.my